AEGIS

Adaptive Exam Guardian & Integrity System

A **browser-native** exam portal that catches cheating without watching students. It reads behaviour as the exam happens, scores it in a way you can explain, and hands the professor **evidence to review**. No webcams, no extensions, no surveillance.

COMP47250 Team Software Project · Microsoft × University College Dublin · MSc Computer Science, 2026 · **Final Presentation**

Vijeth Prashanth Hegde · Lead / Backend & Architecture

Raushan Nerkar · Azure & DevOps / Live Monitor

Tejas Tholakalabavi Dinesh · Frontend / Exam Shell

Lakshmi Kiran C M · Auth & Signal Scoring

Jeevika Bangalore Thanuj Kumar · Telemetry SDK / Scorer

Tejas Pantharapalya Venkatesh · Database & Data

▶ Live on Azure `web.aca.aegisexam.app` Container Apps • `web.aks.aegisexam.app` AKS

Generative AI broke online exams, and the usual fixes don't fit

A student can leave the exam tab, ask an LLM for a clean answer, and paste it back in seconds. Universities are left choosing between two bad options.

Invasive proctoring

Tools like Proctorio, Honorlock, and Respondus lean on webcams, screen capture, and lockdown browsers. They are intrusive, fiddly to set up, and the biometrics they grab fall under GDPR Article 9.

Post-hoc AI detectors

These classifiers only read the finished answer, never how it was written. They are unreliable, quick to false-positive, and leave you nothing defensible about *how* the work was produced.

OUR SOLUTION · THREE PILLARS

AEGIS watches *how* an answer gets written, not what is on the screen

1 · Behavioural telemetry

Six lightweight browser signals stream live over a WebSocket: tab visibility, paste metadata, keystroke intervals, focus and blur, answer timing, and viewport size. Nothing beyond that.

2 · Explainable scoring

A weighted scorer turns the six signals into one 0 to 1 number, and you can see **each signal's share** of it. A z-score typing baseline anchors the maths, and no single event is ever enough to flag someone.

3 · Human-reviewed report

The professor gets a colour-coded risk score and a timeline of flagged moments to read as **evidence**, then makes the call. AEGIS never punishes anyone itself.

USER STORIES · TWO CONTEXTS

Two users, two needs, one system that balances them

👩‍🏫 Dr Sarah, the professor

- Builds a quiz, sets it **closed- or open-book**, picks a sensitivity preset, and schedules it.
- Watches a **live integrity console** where each student's risk card refreshes every five seconds.
- Once it is over, she grades short answers (with optional **AI grade suggestions**), reads the **flagged-event timeline**, and releases results. Enrolled students get an email automatically.
- What she needs: evidence she can defend, not a black box.

👨‍🎓 James, the student

- Reads a plain-English **consent notice** that spells out exactly what is and isn't collected.
- Sits the exam in a focused shell: one question at a time, a countdown, auto-submit, and a progress sidebar.
- Accidentally switches tab once. It is **logged, not punished**, and a quiet banner lets him know.
- What he needs: a fair, open exam with no webcam and no heavy lockdown.

There is also a **super-admin** who manages users and keeps an eye on a tamper-evident audit log across the whole institution.

STATE OF THE ART · WHERE AEGIS FITS

The privacy-first spot the big players leave empty

CAPABILITY	PROCTORIO / HONORLOCK	RESPONDUS LOCKDOWN	AEGIS
Monitoring method	Webcam + screen recording	Browser lockdown	**Behavioural metadata only**
Student privacy	**Biometric capture**	Restrictive environment	**Data-minimised, no biometrics**
Install footprint	Extension / app	Dedicated browser	**None, any modern browser**
Output	Video for manual review	Block-based prevention	**Explainable per-signal score**
Stance	Surveillance / prevention		**Evidence for human judgement**

📘 **Non-goals we chose on purpose:** no webcam or mic, no clipboard scraping, no keystroke logging, no hard lockdown. Each one would break our whole point, **privacy-preserving evidence without surveillance**, so we left them out deliberately and wrote down why.

Three planes, and monitoring can never block the exam

Client · React + TS

Exam shell | Telemetry SDK | Professor console | Admin

Backend · FastAPI

REST (71 routes) | WebSocket gateway | Signal scorer | Live monitor | AI copilot

Azure platform

PostgreSQL | Service Bus | Blob | Key Vault | Azure OpenAI | Comms (ACS)

The critical design choice

Answers ⟶ **REST POST** ⟶ committed to PostgreSQL
Telemetry ⟶ **WebSocket** ⟶ best-effort, non-blocking

If the socket drops or the scorer lags, the student still submits. We built fairness into the structure instead of hoping for it.

TECHNOLOGY STACK · JUSTIFIED CHOICES

Every layer picked over an alternative we turned down

LAYER	CHOSEN	WHY	REJECTED
Backend	**FastAPI · Py 3.12**	ASGI-native async + WebSocket on one loop; Pydantic v2	Django (sync), Flask (no async/WS)
Frontend	**React 18 + TS**	Typed telemetry payloads caught schema drift at build time	Vue / Angular (ramp-up)
Database	**PostgreSQL + JSONB**	ACID for answers; JSONB for varied events; SQLite in tests	Mongo, TimescaleDB, DynamoDB
Messaging	**Azure Service Bus**	Managed PaaS, dead-letter queue, native KEDA autoscale	Kafka / RabbitMQ (ops)
Compute	**Container Apps + AKS**	One ACR image on both: Container Apps for scale-to-zero, AKS (Helm) for portable Kubernetes	App Service (WS limits)
Scoring	**Rule-based + z-score**	Explainable, no labelled data, tunable via presets	ML/XGBoost (Phase 3)
AI copilot	**Azure OpenAI**	Explainable review overlay; Ollama local twin keeps CI + dev fully offline	Self-hosted LLM only

📓 **12 architecture decisions** live in `docs/DECISIONS.md`, each with the date, the reasoning, the alternatives we weighed, and the consequences.

We generate our own telemetry, and keep it minimal

✓ Collected (metadata)

- Tab visibility timestamps
- Paste event + char delta
- Inter-keystroke interval (ms)
- Focus/blur, resize, answer timing

× Never collected

- Key values / typed content
- Clipboard text
- Screen, webcam, microphone
- Browsing history / other apps

💡 **Consent-gated:** the API returns `403` for questions until we have stored a `consent_at` timestamp. Our legal basis is legitimate interest (Art. 6(1)(f)) plus informed acknowledgement, with a planned 90-day retention window.

KEY TECHNICAL CHALLENGE · 1 OF 3

Real-time telemetry that never blocks a submission

Stream events from hundreds of students at once, and still guarantee that a monitoring hiccup never costs anyone an answer.

The problem

Monitor synchronously and you risk blocking answer saves. A dropped socket can't be allowed to cost a student their exam.

Our contribution

- REST answers commit **first**, before any side-effect runs
- The telemetry WebSocket and Service Bus are best-effort, wrapped in try/except
- The client SDK holds a 500-event ring buffer, reconnects with exponential backoff (1s to 30s), and replays on reconnect

KEY TECHNICAL CHALLENGE · 2 OF 3

Explainable scoring with real statistics, not a black box

Six weighted signals become a 0 to 1 score (standard preset)

📐 Formal statistical analysis

We build each student a typing **baseline** (median IKI and standard deviation) from their first few minutes, then score live windows against it with a **z-score**. That covers the module's formal-statistics requirement, in both the scoring and the AUC evaluation.

KEY TECHNICAL CHALLENGE · 3 OF 3

Privacy-by-design telemetry that still carries signal

Pull a useful behavioural signal out of the **least data we can get away with**. That constraint is both an engineering problem and an ethical one.

Timing, not content

The SDK reads `keydown` timestamps and throws away the key itself in the browser. We keep the *gaps between keystrokes*, never the characters. This is not a keylogger.

Paste size, not text

A paste logs how much the input grew, and we never touch the clipboard. That is enough to catch a 600-character dump without keeping anything sensitive.

Consent + minimisation

A required notice before the exam, metadata-only storage, no biometrics (so GDPR Art. 9 does not apply), and human-review-only use (which satisfies Art. 22).

An Azure OpenAI copilot that helps a professor review, and never replaces them

Three on-demand helpers sit on top of the deterministic scorer. Everything they produce is a suggestion the professor accepts, edits, or throws out. Nothing saves itself.

1 · Integrity Brief

Reads the six sub-scores and event counts and writes a plain-English summary. It sees **metadata only**, no keystrokes or answer text, and always ends with a not-a-verdict disclaimer.

2 · Grade Assist

For each short answer it proposes a score, a one-line reason, and a confidence, judged against the model answer and rubric. The professor hits **Accept** or types a different mark. Nothing saves on its own.

CURRENT STATUS · ENGINEERING PRACTICES

Industry practice, with the receipts, on a system that is actually deployed

370
backend tests (40 files)

79
frontend tests + Playwright E2E

71 + 2
REST + WebSocket endpoints

v1.0
tagged & live on Azure

⚙️ CI/CD & quality gates

- Every PR runs `ci.yml`: ruff, pyright, pytest, ESLint, tsc, and Playwright E2E
- `cd.yml` ships to Container Apps and `deploy-aks.yml` ships to AKS via Helm; both build, push to ACR, and smoke-test
- A protected `main`, required peer review, Conventional Commits

A controlled pilot cleanly separated honest work from assisted

Protocol (n = 10, 5 honest / 5 AI-assisted)

In the assisted runs we combined tab-switching **with** a paste over 100 characters, while the honest runs typed everything from scratch. Both used the live `standard` preset with a flag threshold of 0.40.

	PRED. FLAGGED	PRED. CLEAR
Assisted	**TP = 5**	FN = 0
Honest	FP = 0	**TN = 5**

1.00
Precision (≥0.80)

1.00
Recall (≥0.75)

0.00
False-positive rate (≤0.10)

1.00
AUC-ROC (≥0.80)

Tested under load, under failure, and with real users

⚡ Performance (k6, 50 & 100 students)

METRIC	50	100
WS connection success	**100%**	**100%**
REST answer p99 (≤800ms)	**390 to 545ms**	**686 to 952ms***
Backend memory	**172 MiB**	**173 MiB**
WS 5xx errors	**0**	**0**

*One worker under a synchronised burst; the median was around 545ms and nothing failed. Adding replicas flattens the tail.

🧪 System correctness

- WS-failure isolation → answer still submits ✓
- State machine draft→open→closed enforced ✓
- Consent gate returns 403 before questions ✓

Does AEGIS solve the problem it set out to?

Yes, within the scope we set: a working, deployed, privacy-preserving system that turns exam behaviour into evidence a human can read and explain.

Strengths

- End to end and live on Azure, on both Container Apps and AKS
- A non-blocking, resilient real-time pipeline
- Explainable scoring backed by z-score statistics
- An Azure OpenAI copilot that stays an evidence-only overlay
- Privacy-first, a niche few others touch
- Deep test coverage and full CI/CD

Each limitation has a concrete next step

🤖 Smarter scoring

- ML overlay (XGBoost) on the six features, trained on pilot labels
- Per-student baseline calibration to cut IKI false positives
- Extend the AI copilot's collusion analytics beyond short answers

📈 Scale & resilience

- Move live state into Redis and fan telemetry out across replicas
- Durable Service-Bus scoring worker with retry/DLQ
- Provisioned Azure OpenAI throughput + PostgreSQL HA for the load window

✨ Polish & ops

- App Insights dashboards + alerts (near-free on credit)
- Question randomisation & per-student pools
- Autoscaling policies (KEDA / HPA) tuned per host

Shipped since the MVP: custom domains and managed TLS on both hosts, AKS delivery via Helm, the Azure OpenAI copilot, and email for password resets and grade-ready alerts through Azure Communication Services. All of it fits comfortably inside the €800 Azure credit.

TEAM MANAGEMENT & PROCESS

Six owners, a full trimester of evidence, and feedback that changed the build

Ownership

Vijeth	Backend architecture, WS gateway, E2E
Raushan	Azure IaC, CI/CD, live monitor, admin
Tejas TD	Exam shell, auth UI, timer, history
Lakshmi	Auth/RBAC, scoring components, presets
Jeevika	Telemetry SDK, scorer engine, baseline
Tejas PV	DB schema, migrations, seed, CSV, ADR

108
Jira tickets Done

6/6
members committing all trimester

What we built, what we learned, and how AI shaped it

🧭 Lessons & teamwork

- Cutting scope, pushing ML and webcams to Phase 3, is what made the MVP **real and defensible**
- Working contract-first, with OpenAPI and typed payloads, let six of us build in parallel
- Next time we would design shared state for multiple replicas from day one

🤝 AI in practice

- AI helped most with scaffolding, IaC and Bicep, test fixtures, docs, and debugging
- We kept architecture, evaluation design, and scorer logic in our own hands
- Our rule: **no AI-written code merges unless the author can explain every line**, and it is checked in review

AEGIS is substantial, built with the ethics in mind, and scoped to actually ship: minimal telemetry, scoring you can explain, and a report that backs a professor's judgement.

▶ Live demo `web.aca.aegisexam.app` Container Apps • `web.aks.aegisexam.app` AKS